



RAPPORT DE STAGE

Stage effectué chez

SPLAYCE

42 Route d'Abbaretz, 44170 Nozay

Réalisé par :

Nora MERHAND

Remerciements

Je tiens à remercier l'entreprise Splayce pour m'avoir accueillie au sein de son équipe dans le cadre de mon stage de deuxième année de BTS Services Informatiques aux Organisations. Cette expérience m'a permis de développer mes compétences techniques tout en découvrant le fonctionnement de projets en entreprise.

Je remercie tout particulièrement Cyriac pour son aide technique, ses explications et les échanges que nous avons eus, qui m'ont permis d'approfondir mes connaissances et de gagner en autonomie.

Je remercie également Camille, cheffe de projet, pour son accompagnement, sa confiance et ses retours constructifs. Sa supervision m'a permis de mieux comprendre les exigences d'un projet et d'améliorer la qualité de mon travail.

Je tiens aussi à remercier Erwan, mon tuteur, pour avoir pris le temps de m'accueillir dans les meilleures conditions et d'avoir facilité mon intégration dans l'équipe.

Enfin, je remercie l'ensemble de l'équipe pour son accueil et sa bienveillance, ainsi que mes enseignants pour leur encadrement durant cette formation.

Sommaire

INTRODUCTION.....	1
CONTEXTE	1
PRESENTATION DE L'ENTREPRISE	1
CONDITIONS D'ACCUEIL	1
TECHNOLOGIES UTILISEES.....	2
MISSIONS	2
CATALOGUE DIGITAL SPLAYCE	3
LUMMEA.....	5
COMPETENCES ACQUISES.....	12
COMPETENCES TECHNIQUES	12
COMPETENCES ORGANISATIONNELLES	12
COMPETENCES RELATIONNELLES	12
BILAN TECHNIQUE	13
TRAVAUX REALISES	13
TRAVAUX EN COURS.....	13
CONCLUSION	13

Introduction

Dans le cadre de ma deuxième année de BTS Services Informatiques aux Organisations, j'ai réalisé un stage au sein de l'entreprise Splayce, située à Nozay et faisant partie de GDCom Group. Ce stage s'inscrit dans la continuité de ma formation et avait pour objectif d'acquérir de l'expérience dans le domaine du développement en milieu professionnel.

Ce rapport présentera tout d'abord le contexte du stage, avec une rapide présentation de l'entreprise, puis les missions que j'ai réalisées, les compétences que j'ai pu développer, un bilan technique, et enfin une conclusion sur cette expérience professionnelle, mettant en lumière ce que ce stage m'a apporté tant sur le plan technique que personnel.

Contexte

Présentation de l'entreprise

L'entreprise Splayce a été créée en 1992. Elle est spécialisée dans le secteur de la santé et propose à ses clients des stratégies de communication sur-mesure. Son siège social est situé à Nozay, bien que l'entreprise dispose de 5 sites au total. Aujourd'hui, elle compte environ 116 employés.

L'organisation interne de Splayce est structurée autour de plusieurs départements principaux : développement produit, design, marketing, commerce, relation client et production. J'ai effectué mon stage au sein du service IT, qui est chargé de la gestion et de la maintenance de l'ensemble des systèmes informatiques de l'entreprise. Ce service joue un rôle essentiel dans l'organisation, car il assure le bon fonctionnement des outils utilisés par tous les autres services et garantit la sécurité des données. Il assure également le déploiement de nouvelles solutions technologiques pour répondre aux besoins croissants de l'entreprise et accompagner sa croissance.

Splayce accorde une grande importance à l'innovation et à l'adaptation aux évolutions du secteur de la santé. Les différents services collaborent étroitement pour développer des stratégies efficaces et personnalisées pour leurs clients. La culture d'entreprise encourage la communication, le partage de connaissances et le travail en équipe, ce qui permet de maintenir un environnement professionnel dynamique et réactif.

Conditions d'accueil

À mon arrivée dans l'entreprise, on m'a fourni un ordinateur accompagné d'une souris, d'un écran, d'un casque audio, d'un chargeur et d'une sacoche de rangement. J'ai également reçu mes codes de session, comprenant une adresse électronique et un mot de passe, ainsi que le code d'accès pour le portail principal et le bureau du service IT. Le service IT est installé dans un grand open-space où tous les membres travaillent ensemble.

Lors de ma première matinée, j'ai rencontré les différents membres de l'équipe. J'ai principalement été présentée à Cyriac, un des développeurs avec qui je travaillerai tout au long de mon stage. Il m'a ensuite aidée à installer mon environnement de développement, incluant VSCode, GitBash, l'outil nécessaire pour communiquer avec GitHub, et Node.js. Puis, il m'a fait visiter l'entreprise en me présentant les différents services et salariés.

Progressivement, j'ai fait connaissance avec le reste de l'équipe IT, composée de sept membres, incluant des développeurs, des administrateurs réseaux, une cheffe de projet et le DSI. Durant mon stage, j'ai principalement collaboré avec Cyriac et Camille, la cheffe de projet.

J'ai été très bien accueillie et intégrée, ce qui m'a permis de me familiariser rapidement avec les outils et les méthodes de travail de l'équipe. Bien que ma deuxième journée se soit déroulée en télétravail en raison de la neige, j'ai rapidement pu commencer à travailler sur mes premières missions.

Technologies utilisées

Durant mon stage, j'ai travaillé sur deux outils principaux, chacun ayant ses spécificités techniques et fonctionnelles.

Catalogue digital Splayce

Le catalogue digital est un outil d'aide à la vente hors ligne destiné aux commerciaux, leur permettant d'accéder aux informations produits même sans connexion internet. Les produits y sont organisés par catégories, peuvent être recherchés via une barre de recherche et sont mis à jour rapidement grâce à un système de synchronisation avec les données Salesforce. Le projet a été mis en production le 5 janvier 2026.

Technologies utilisées :

- Front-end : HTML, TypeScript, React, Tailwind CSS pour la création d'interfaces interactives et réactives
- Back-end : Node.js pour la logique serveur
- Bases de données : Salesforce (SOQL) pour les données présentes sur le CRM et Dexie pour la base locale
- Outils de collaboration : GitHub pour la gestion des versions et travail collaboratif, VS Code comme IDE principal et Teams pour la communication

Lumméa

Lumméa est un logiciel de pilotage de contenus pour croix lumineuses et écrans, utilisé par des communes et villes pour de l'affichage dynamique à destination de clients externes. Il fonctionne entièrement en full web et permet de gérer la diffusion de contenus ainsi que la planification des programmes.

Technologies et outils :

- Front-end : React, TypeScript, Tailwind CSS pour la création d'interfaces interactives et réactives
- Back-end / moteur : Node.js pour la logique serveur et Electron pour le moteur
- Bases de données : Prisma pour interagir avec la base Lumméa
- Outils de collaboration : GitHub pour le versioning, VS Code comme IDE principal, Teams pour la communication et Monday pour le suivi des tâches

Missions

Au cours de mon stage, j'ai travaillé sur plusieurs projets aux objectifs et aux contextes variés, ce qui m'a permis de découvrir différents aspects du développement logiciel au sein de l'entreprise. Ces projets répondent à des besoins concrets, aussi bien internes qu'orientés vers les clients, et s'inscrivent dans une logique d'amélioration des outils existants et de création de nouvelles solutions numériques.

J'ai principalement participé à deux projets majeurs :

- Le premier concerne un catalogue digital hors ligne destiné aux commerciaux de l'entreprise.
- Le second porte sur Lumméa, un logiciel permettant de gérer l'affichage d'informations sur les écrans de mairie ainsi que sur les croix de pharmacie.

Ces deux projets m'ont permis de travailler sur des usages très différents, tout en mobilisant des compétences techniques communes.

Catalogue digital Splayce

Le catalogue est un outil destiné au service commercial de l'entreprise. Il s'agit d'un développement réalisé en interne, dont l'objectif est de faciliter l'accès aux produits proposés par Splayce.

Ce catalogue fonctionne hors ligne afin de permettre aux commerciaux de consulter les informations produits à tout moment, notamment lors de leurs déplacements ou de rendez-vous clients, même en l'absence de connexion internet.

Grâce à cet outil, les commerciaux disposent d'une interface centralisée et pratique pour présenter les produits, leurs caractéristiques et leurs visuels. Le projet vise ainsi à améliorer leur efficacité, à fluidifier la relation client et à offrir un support de vente fiable et facilement accessible.

Gestion des tarifs

Sur ce projet, ma mission principale a été de développer la gestion des tarifs. En effet, lorsque je suis arrivée, les tarifs n'étaient pas disponibles directement sur le catalogue. Les commerciaux devaient alors se référer à un fichier PDF afin de connaître les tarifs de chaque produit.

Les tarifs, comme les autres données présentes dans le catalogue, sont stockés dans Salesforce. J'ai donc commencé par travailler sur la récupération de ces données depuis le CRM. Pour cela, j'ai eu une réunion avec Cyriac et Jérôme, le spécialiste Salesforce, afin que ce dernier me présente l'organisation des données tarifaires dans Salesforce et m'explique comment les récupérer à l'aide de requêtes SOQL.

J'ai ensuite étudié le code existant en profondeur. Cette analyse m'a permis de comprendre que les données sont d'abord récupérées via des requêtes SOQL, puis transformées en JSON, avant d'être finalement compressées dans un dataset. J'ai appliqué cette même logique pour les tarifs, ce qui m'a permis d'obtenir un fichier tarifs.json comme illustré dans l'[Annexe 1](#).

La principale difficulté a concerné la liaison entre les tarifs et leurs produits. En effet, le catalogue distingue des produits parents et des produits enfants. Les produits parents ne possèdent pas de tarifs, contrairement aux produits enfants, qui peuvent en posséder plusieurs. De plus, la référence unique utilisée pour identifier un produit dans le catalogue n'est pas la même que celle utilisée dans Salesforce. J'ai donc dû attribuer un identifiant unique à chaque tarif, ainsi qu'un produit_id représentant le produit enfant auquel le tarif est rattaché.

Dans un premier temps, j'ai testé ma solution sans la récupération des images afin de gagner du temps, le traitement complet avec les images prenant environ une heure. J'ai ensuite adapté ma solution pour inclure les images, ce qui m'a permis d'obtenir un dataset complet.

Une fois les données correctement récupérées depuis Salesforce, j'ai commencé à travailler sur la partie front du projet. J'ai tout d'abord créé une table dédiée aux tarifs dans la base de données locale (Dixie). J'ai ensuite mis en place une mise à jour permettant de remplir cette table à partir des données issues du dataset.

J'ai ensuite travaillé sur l'affichage des tarifs dans le catalogue. J'ai commencé par gérer l'affichage conditionnel de la grille tarifaire. En effet, puisque les produits parents ne possèdent pas de tarifs, l'onglet correspondant n'a pas vocation à s'afficher pour eux. Il n'est donc visible

que pour les produits enfants comme illustré dans les [Annexes 2 et 3](#). À l'intérieur de cet onglet, les tarifs sont affichés sous forme de grille dans une première version fonctionnelle comme illustré dans l'[Annexe 4](#). L'aspect visuel sera retravaillé ultérieurement, une fois les maquettes finalisées.

Je me suis ensuite penchée sur la synchronisation. Le catalogue dispose en effet d'une fonctionnalité permettant de mettre à jour uniquement les éléments modifiés, sans avoir à relancer une mise à jour complète. J'ai donc adapté cette logique pour y intégrer les tarifs. J'ai également ajouté une pop-up rappelant à l'utilisateur de réaliser une synchronisation après chaque samedi, moment où les tarifs sont mis à jour.

Parallèlement, plusieurs réunions ont eu lieu afin de définir précisément les besoins des commerciaux, notamment concernant les champs à afficher. Il a été décidé que les types de prix et le prix du cliché (pour les premiers clients) étaient essentiels. Comme les informations sur les types de prix n'étaient pas initialement présentes dans Salesforce, j'ai dû attendre que Jérôme remonte ces données depuis Silog. J'ai ensuite adapté l'ensemble de ma logique pour intégrer ces nouveaux champs.

Enfin, j'ai réalisé une deuxième version de la grille tarifaire, voir l'[Annexe 5](#), plus proche de celle que les commerciaux utilisent déjà.

Déclinaisons

Sur le catalogue, les produits enfants sont appelés « déclinaisons ». Jusqu'alors, le système permettant d'y accéder était rudimentaire : une simple zone affichant la liste des déclinaisons sous forme de boutons. Une refonte de cet affichage était prévue depuis quelque temps afin d'en améliorer l'ergonomie et l'expérience utilisateur. Des maquettes ont donc été créées à cet effet, voir l'[Annexe 6](#).

Lorsque Camille les a reçues, elle me les a présentées et m'a confié leur développement. J'ai commencé par analyser le code existant afin d'estimer le temps nécessaire à la réalisation. Avec l'aide de Cyriac, nous avons découpé le travail en plusieurs étapes et déterminé leur ordre d'exécution, ce qui m'a permis de produire une estimation à 18h. Après validation par Camille, j'ai pu débuter le développement.

En suivant les étapes définies, j'ai d'abord développé le bouton permettant d'ouvrir une modale latérale contenant la liste des déclinaisons. Ce bouton a été positionné à l'emplacement de l'ancienne liste, en respectant le visuel de la maquette, comme illustré dans l'[Annexe 7](#). J'ai ensuite créé une modale vide et mis en place une animation fluide d'ouverture et de fermeture depuis le bord gauche de l'écran.

J'ai poursuivi avec la création du header de la modale, comprenant le titre « Déclinaisons », le nombre de déclinaisons associées au produit ainsi qu'une icône de fermeture. J'ai ensuite implémenté le mapping des déclinaisons sous forme de liste. Dans un premier temps, seule la référence du produit était affichée, puis j'y ai ajouté le nom, et enfin la photo. Afin d'assurer un rendu homogène, j'ai contraint la taille des images dans un cadre à fond blanc, car toutes ne sont pas détournées et leurs dimensions varient, notamment en hauteur, voir l'[Annexe 8](#).

Une fois l'affichage terminé, j'ai mis en place la sélection d'une déclinaison. Lorsqu'un utilisateur en sélectionne une, sa fiche produit s'affiche correctement et la modale se ferme automatiquement.

Nous avons ensuite longuement échangé avec Cyriac concernant l'indication visuelle d'une déclinaison sélectionnée. Cet élément n'étant pas précisé dans la maquette, nous avons réfléchi à la meilleure manière de la mettre en évidence tout en respectant la charte graphique, voir l'[Annexe 9](#) pour le visuel que nous avons choisi.

J'ai ensuite développé une barre de recherche permettant de filtrer la liste des déclinaisons par référence ou par nom. Par défaut, le filtre est positionné sur « Tous ». L'utilisateur peut ensuite choisir « Nom du produit » ou « Référence », ce qui paramètre la barre de recherche pour filtrer uniquement selon le critère sélectionné. Lorsque le filtre « Nom du produit » est activé, l'ordre

d'affichage est également adapté en inversant la position du nom et de la référence dans la liste, comme illustré dans les [Annexes 8](#) et [10](#).

Après l'implémentation des fonctionnalités, j'ai effectué plusieurs ajustements visuels afin de respecter au mieux la maquette et la charte graphique. J'ai ensuite présenté mon travail à Cyriac puis à Camille, qui l'ont validé. La nouvelle gestion des déclinaisons a également été présentée à plusieurs membres du pôle développement produit ainsi qu'à la responsable marketing, qui dit « adorer ».

J'ai mis au total 7h à développer l'ensemble de la solution, soit 11h de moins que prévu. La mise en production a été réalisée le 12 février 2026.

Lumméa

Lumméa est un logiciel permettant de gérer les différents écrans d'affichage présents dans les pharmacies, notamment les croix lumineuses. Il est utilisé pour diffuser des informations visibles par le public, comme des messages informatifs, des annonces ou des communications institutionnelles.

Sur ce logiciel, on m'a confié plusieurs missions, de durées et d'importances variées, allant de petites interventions ciblées à des tâches plus conséquentes. Ces missions m'ont permis de travailler sur différents aspects du logiciel et de mieux comprendre son fonctionnement global, tout en m'adaptant aux besoins spécifiques des utilisateurs finaux.

Afin de mieux comprendre l'organisation du logiciel et les responsabilités associées à chaque profil, il est important de présenter les différents types d'utilisateurs ainsi que leurs droits d'accès. Lumméa fonctionne selon un système de rôles, permettant de restreindre ou d'élargir l'accès aux différentes fonctionnalités (contenus, programmes, planning, administration).

Le tableau ci-dessous synthétise les profils existants et leur champ d'action au sein de l'application :

Intitulé	Exemple – qui ?	Contenus	Programmes	Planning	Administration	Champ d'action
Super administrateur	Service IT	X	X	X	X	Tous les comptes
Revendeur	Revendeur panneau / croix de pharmacie	X	X	X	X Simplifié	Owner spécifique
Administrateur	Titulaire d'une pharmacie	X	X	X		Owner spécifique
Utilisateur	Préparateur en pharmacie	X	X			Owner spécifique
Créateur de modèle	Camille	X				

Visualisation des mots de passes

J'ai commencé par intervenir sur la page de connexion du logiciel, où un besoin précis avait été identifié : permettre à l'utilisateur de visualiser le mot de passe saisi grâce à une icône en forme d'œil. J'ai donc été chargée de développer cette fonctionnalité.

J'ai commencé par échanger avec la cheffe de projet afin de choisir l'icône la plus adaptée. Une fois ce choix validé, j'ai utilisé React pour mettre en place une vue conditionnelle du mot de passe, voir l'[Annexe 11](#). Lors du clic sur l'icône, le champ de saisie alterne entre un type *password* (mot de passe masqué) et un type *text* (mot de passe visible), comme illustré dans l'[Annexe 12](#).

Une fois la solution fonctionnelle, j'ai envoyé le développement en préproduction afin que Camille puisse effectuer les tests nécessaires et me faire un retour. N'ayant reçu aucune remarque, la fonctionnalité a ensuite été mise en production avec d'autres fonctionnalités le mercredi 21/01/2026.

Barre de scroll

Sur Lumméa, il est possible d'éditer des « contenus », c'est-à-dire les éléments qui sont ensuite intégrés dans des « programmes » afin d'être affichés sur les écrans. Dans la section d'édition des contenus, certaines parties nécessitent de faire défiler la page pour accéder à l'ensemble des options. Cependant, aucune barre de défilement ni indication visuelle n'était présente pour faciliter ce scroll.

Un retour a donc été formulé afin de rendre le défilement plus visible. Lors des échanges, l'idée d'utiliser des « ancres » a été envisagée pour des raisons esthétiques. Toutefois, cette solution ne garantissait pas l'accès à l'ensemble des éléments de la page, notamment ceux situés entre deux ancres. Le choix final s'est donc porté sur l'ajout d'une barre de défilement simple à affichage conditionnel.

Cette barre de scroll n'apparaît que lorsque le défilement est nécessaire et s'adapte à la taille de l'écran. Si la taille de l'écran change et que le scroll n'est plus requis, la barre disparaît automatiquement. Une fois la fonctionnalité développée, je l'ai envoyée en préproduction afin que Camille puisse effectuer les tests nécessaires et me faire un retour. À la suite de ces retours, j'ai ajusté les couleurs de la barre de défilement afin qu'elle respecte la charte graphique du logiciel, voir l'[Annexe 13](#).

Après ces modifications, la fonctionnalité a été mise en production le mercredi 21/01/2026, en même temps que d'autres évolutions.

Tri des programmes

Sur la page « contenus », il existe une fonctionnalité de tri des éléments selon quatre critères : Nom de A à Z, Nom de Z à A, Plus récent et Plus ancien. Cette fonctionnalité devait également être disponible sur la page programmes, mais les boutons prévus à cet effet ne fonctionnaient pas.

En investiguant, j'ai découvert que la page programmes possédait en réalité une fonctionnalité de tri identique à celle de la page contenus, mais qu'elle n'était jamais activée. J'ai donc relié les boutons de la page programmes à cette fonctionnalité. Après tests, le tri fonctionnait parfaitement.

La modification a ensuite été déployée en préproduction et, n'ayant reçu aucun retour, elle a été mise en production en même temps que les autres fonctionnalités, le 21/01/2026.

Barre de recherche

Sur la page administration, il est possible d'accéder à la section « Lecteurs », qui correspond aux différents écrans accessibles par l'administrateur ou le revendeur connecté. Un revendeur n'a accès qu'aux écrans rattachés aux entités de ses clients ainsi qu'à sa propre entité.

Depuis cette page Lecteurs, il est possible de modifier un écran, notamment de changer son propriétaire, c'est-à-dire l'entité à laquelle l'écran est rattaché. Cette modification se faisait initialement via une liste déroulante simple.

Au départ, cette solution ne posait pas de problème, le nombre d'entités étant limité. Cependant, avec l'augmentation progressive de ce nombre, le besoin d'une barre de

recherche au sein de la liste est rapidement apparu. J'ai donc été chargée de développer cette fonctionnalité.

Après analyse, j'ai constaté que le composant utilisé était un Select. J'ai tenté d'y intégrer une barre de recherche via un champ input, mais je me suis rendu compte qu'il était impossible de donner le focus à un input à l'intérieur de ce type de composant.

J'ai donc choisi de reproduire entièrement le composant, en y intégrant directement une barre de recherche. Une fois le composant fonctionnel, j'ai travaillé sur son apparence afin qu'il soit le plus proche possible du composant d'origine, tant visuellement que fonctionnellement, comme illustré dans les Annexes 14 et 15.

Après déploiement en préproduction et sans retour de Camille, la fonctionnalité a été mise en production le 21/01/2026.

Par la suite, je me suis aperçu que la même liste déroulante existait côté revendeurs, mais qu'elle n'avait pas encore été modifiée. J'ai donc informé Camille et lui ai demandé si je devais m'en occuper. Elle m'a confirmé que c'était le cas, et je me suis donc attelée à cette tâche. J'ai rencontré quelques ajustements à effectuer côté front, ainsi que de petits oublis d'import à corriger lors de la mise en préproduction.

Une fois ces points résolus, la fonctionnalité était prête à être testée par Camille.

Modification des noms des programmes

Actuellement, les programmes sont modifiables dans la mesure où il est possible de changer les contenus qui les composent ainsi que les transitions entre ces contenus. Le besoin exprimé portait sur la possibilité de modifier le nom des programmes préexistants.

Pour répondre à ce besoin, j'ai commencé par rendre la zone d'affichage du nom du programme éditable à l'aide d'une icône en forme de crayon. J'ai ensuite ajouté une icône d'enregistrement, comme illustré dans l'[Annexe 16](#) ainsi que la logique associée afin de permettre la modification du nom du programme directement en base de données via une requête Prisma, voir l'[Annexe 17](#).

Une démonstration a été réalisée auprès de Camille et Cyriac, qui ont validé la fonctionnalité. Celle-ci a donc été envoyée en préproduction après la mise en production du 21/01/2026, afin de permettre des tests plus approfondis.

À la suite de ces tests, plusieurs retours ont été formulés :

- Permettre l'enregistrement de la modification du nom à l'aide de la touche Entrée du clavier ;
- Limiter le nombre de caractères à 30 afin d'éviter des problèmes d'affichage dans le planning ;
- Faire en sorte que le nom du programme soit également modifié pour les programmes déjà planifiés dans le calendrier.

Les deux premières demandes n'ont posé aucun problème et ont pu être mises en œuvre rapidement. En revanche, la dernière demande soulève une difficulté majeure. En effet, la page planning repose sur une logique particulièrement complexe, notamment en ce qui concerne la gestion des données, qui sont stockées dans un JSON transmis au moteur pour être interprété.

Pour permettre la modification du nom des programmes déjà planifiés, il serait nécessaire de revoir entièrement la logique de la page planning, ce qui représenterait un travail de refactorisation conséquent, estimé à plusieurs semaines.

Ces éléments ont été communiqués à Camille, qui a indiqué que cette évolution devait être mise de côté pour le moment, car il s'agit d'un gros ticket non priorisé à court terme. La fonctionnalité a donc été abandonnée temporairement.

Bug de la page planning

Lumméa dispose d'une page planning permettant de paramétrer l'affichage des programmes pour chaque lecteur. Un lecteur correspondant à un écran.

Pour programmer un affichage, l'utilisateur sélectionne un jour, puis choisit un programme dans une liste proposée. Cette liste est filtrée en fonction du type d'écran du lecteur sélectionné.

Un bug avait toutefois été identifié. Lors de l'arrivée sur la page planning, si l'utilisateur tentait d'ajouter un programme à l'écran courant sans changer d'écran, la liste des programmes disponibles était vide. En revanche, dès qu'un écran était explicitement sélectionné (même s'il s'agissait du même écran), la liste des programmes apparaissait correctement.

Après analyse, j'ai constaté l'existence de deux listes de programmes :

- Une liste contenant l'ensemble des programmes ;
- Une liste contenant les programmes filtrés selon la taille/type d'écran.

Le problème venait du fait que la liste des programmes filtrés n'était remplie et mise à jour uniquement lors d'un changement d'écran. Or, cette liste étant vide par défaut au chargement de la page, et aucun changement d'écran n'étant détecté, elle restait vide, empêchant l'affichage des programmes.

Pour corriger ce comportement, j'ai modifié la logique afin que la liste des programmes filtrés soit recalculée :

- À chaque changement d'écran ;
- Et à chaque mise à jour de la liste de tous les programmes.

Ainsi, dès que la liste des programmes est chargée à l'arrivée sur la page, le filtrage est effectué automatiquement. Après cette correction, le bug ne se reproduisait plus.

La résolution de ce bug a toutefois présenté une difficulté : il était impossible de reproduire celui-ci en environnement local car le temps de chargement en local est beaucoup plus rapide qu'en préproduction ou en production. J'ai donc dû effectuer mes tests directement en préproduction, alors vide, afin d'identifier et corriger le problème.

Une fois la correction validée, la version a été laissée en préproduction, testée par Camille, puis approuvée et mise en production le 04/02/2026.

Ajout de nouvelles transitions

Un programme étant composé de plusieurs contenus, il est possible de sélectionner des transitions à insérer entre ceux-ci afin de dynamiser l'affichage. Cependant, certains utilisateurs ont exprimé le souhait de disposer d'un plus grand nombre de transitions, notamment certaines qui étaient présentes dans la version 4.5 de Lumméa. J'ai donc été chargée de les réintégrer dans la version 5.

Initialement, trois transitions devaient être ajoutées : *ouverture horizontale*, *rideau* et *glisse à gauche*. Après analyse du code actuel de Lumméa (v5) et de celui de la version 4.5, j'ai constaté que la logique des transitions avait évolué entre les deux versions.

En effet, dans la v4.5, les transitions étaient positionnées avant les contenus et permettaient donc d'ouvrir le contenu. À l'inverse, dans la v5, les transitions sont placées après les contenus et servent à les fermer. Ainsi, la transition *ouverture horizontale* est devenue *fermeture horizontale*.

J'ai ensuite étudié les transitions *rideau* et *glisse à gauche*. Il s'est avéré que, compte tenu de la logique actuelle des transitions, *rideau* était impossible à implémenter. En revanche, *glisse à gauche* existait déjà sous le nom de *glissement gauche*.

Pour développer la transition *fermeture horizontale*, j'ai dû intervenir directement dans le code du moteur, développé en Electron.

Par la suite, après avoir observé les autres transitions disponibles dans la v4.5, j'en ai sélectionné plusieurs afin de les adapter et de les intégrer à la v5. J'ai notamment développé les transitions suivantes :

- *Fermeture verticale*
- *Aspiration*
- *Coin en bas à droite*
- *Coin en bas à gauche*
- *Coin en haut à droite*
- *Coin en haut à gauche*
- *Grille losange*
- *Rotation horizontale*
- *Glissement bas*
- *Glissement droite*
- *Papillon envol bas*
- *Papillon envol droite*

J'ai également procédé à un renommage de certaines transitions afin d'assurer une meilleure cohérence et correspondance avec leur fonctionnement réel. Par exemple :

- *Grille* est devenue *grille carré* afin de la différencier de *grille losange* ;
- *Rotation verticale* est devenue *rotation* ;
- *Rotation horizontale* est devenue *rotation verticale* ;
- *Fermeture horizontale* et *fermeture verticale* ont été inversées afin de correspondre au sens des transitions *rotations*.

Enfin, j'ai réorganisé les transitions par ordre alphabétique, effectué l'ensemble des traductions nécessaires et ajouté des commentaires dans le code afin d'en améliorer la lisibilité et la maintenabilité.

Copie des contenus

Les revendeurs n'ont accès qu'aux comptes de leurs clients. Ils peuvent gérer leur propre entité, créer et administrer des entités clientes, attribuer des lecteurs (écrans) aux entités et créer des contenus pour leurs clients, à condition de les concevoir directement dans l'entité cliente concernée. Ils souhaitaient toutefois pouvoir créer des contenus dans leur propre entité afin de les dupliquer ensuite vers les entités de leurs clients.

J'ai donc été chargée de développer une solution de duplication de contenus du compte revendeur vers un compte client sélectionné, à partir d'un cahier des charges fourni.

Dans une première version de la fonctionnalité, une modale accessible depuis la page « Contenus » des comptes revendeurs et super administrateurs permettait de sélectionner le client cible ainsi que les contenus à dupliquer, voir l'[Annexe 18](#). La liste des contenus était générée dynamiquement afin de n'afficher que les contenus du compte revendeur dont le type correspondait aux types d'écrans disponibles sur le compte client sélectionné. Cette liste était alimentée à l'aide d'une requête Prisma et grisée si aucun compte client n'était sélectionné. Une seconde modale permettait ensuite de résumer les choix effectués (client et contenus) et de confirmer la duplication, voir l'[Annexe 19](#).

Après une réunion de présentation avec Camille et Cyriac, plusieurs axes d'amélioration ont été définis. Il a été décidé de découper le processus en trois étapes distinctes, chacune représentée par une modale :

- Une première pour sélectionner le client cible ([Annexe 20](#))

- Une deuxième pour sélectionner les contenus à copier ([Annexe 21](#))
- Une troisième pour confirmer les choix effectués.

Les textes présents dans les modales ont également été revus et modifiés par Camille. Dans la fenêtre de confirmation, j'ai ajouté un visuel des contenus sélectionnés, comme illustré dans l'[Annexe 22](#). De plus, il est désormais impossible de passer à l'étape suivante sans avoir sélectionné un client cible puis au moins un contenu à copier, voir l'[Annexe 23](#).

Après cette évolution, j'ai présenté l'avancée du travail à Cyriac, qui m'a recommandé d'ajouter une fenêtre de confirmation lors de la fermeture de la modale par clic en dehors de celle-ci. En effet, les utilisateurs de Lumméa n'étant pas toujours à l'aise avec les outils informatiques, il était important d'éviter toute perte accidentelle des sélections en cours, celles-ci n'étant pas enregistrées automatiquement. Cette amélioration a été validée par Camille et intégrée à la fonctionnalité, voir l'[Annexe 24](#).

Une fois le développement finalisé, la fonctionnalité a été envoyée en préproduction afin de réaliser les tests. Un retour a alors permis de mettre en évidence un problème lors de la sélection des contenus : un seul type d'écran était sélectionnable, alors que tous les types d'écran compatibles avec le client cible auraient dû l'être.

J'ai corrigé la requête afin que les contenus proposés correspondent correctement à l'ensemble des écrans acceptés par le client cible. Après correction, la fonctionnalité a été redéployée en préproduction.

À la suite de cela, j'ai reçu un nouveau retour de Camille, qui souhaitait qu'une pop-up informe l'utilisateur du succès de la duplication. De mon côté, j'ai identifié deux améliorations possibles pour l'expérience utilisateur :

- Un bouton « Tout désélectionner », permettant à l'utilisateur de désélectionner rapidement tous les contenus sélectionnés sans avoir à les décocher un par un.
- Un filtre par type d'écran, permettant de visualiser uniquement les contenus d'un même type d'écran.

Ces deux idées ont été validées par Camille et Cyriac. Ce dernier m'a toutefois indiqué de prévoir un bouton « Tout sélectionner » si un bouton « Tout désélectionner » était présent.

J'ai donc commencé par développer la pop-up, en utilisant le système déjà existant pour cette fonctionnalité. La pop-up affiche un message de succès si la duplication s'effectue correctement, et un message d'échec en cas de problème, avec la traduction correspondante, voir l'[Annexe 25](#).

Ensuite, j'ai développé le bouton « Tout sélectionner / Tout désélectionner », initialement placé au-dessus de la liste déroulante. N'étant pas convaincu par cet emplacement, j'ai demandé l'avis de Camille et Cyriac, qui a proposé de le placer en haut de la liste déroulante, ce qui s'est avéré très bien intégré, comme illustré dans l'[Annexe 26](#).

Pour le filtre par type d'écran, j'ai modifié ma requête Prisma, voir [Annexe 27](#), afin de récupérer le type d'écran (ID et nom) et j'ai rempli la liste déroulante avec ces informations. J'ai ajouté le type d'écran à côté du nom dans la liste pour fournir plus d'indications à l'utilisateur, puis j'ai développé la logique de filtrage en adaptant tous les autres éléments de la liste. Enfin, j'ai ajusté le style de la liste déroulante (taille, curseur, etc.), voir l'[Annexe 28](#).

Une fois ces modifications réalisées, j'ai redéployé la fonctionnalité en préproduction afin de procéder à une nouvelle phase de tests.

Un léger ajustement a été demandé concernant l'affichage du titre et du sous-titre : sur les très grands écrans, ceux-ci apparaissaient côte à côte au lieu d'être positionnés l'un au-dessus de l'autre. J'ai donc corrigé ce comportement afin d'assurer un affichage cohérent sur tous les formats d'écran.

Après cette correction, la fonctionnalité a été de nouveau déployée en préproduction, où elle a finalement été validée.

Affichage des programmes prioritaires sur le planning

Les programmes prioritaires sont, comme leur nom l'indique, des programmes qui ont la priorité sur les programmes classiques. Autrement dit, si un programme classique et un programme prioritaire sont programmés en même temps sur le planning, c'est le programme prioritaire qui sera affiché. Cette fonctionnalité a été développée par Cyriac à la demande des utilisateurs.

Cependant, les programmes prioritaires ne s'affichaient pas sur le planning, ce qui faisait que les utilisateurs ne comprenaient pas toujours pourquoi un programme qu'ils n'avaient pas choisi apparaissait sur leur écran : il s'agissait en fait d'un programme prioritaire. Il m'a donc été demandé de signaler les programmes prioritaires sur le planning.

À l'origine, il était prévu de simplement afficher une pastille de visualisation, mais après discussion et réflexion, il a été décidé d'afficher directement le nom des programmes prioritaires. Il fallait toutefois les différencier des programmes classiques. Camille a demandé qu'une maquette soit réalisée pour cela. En attendant de la recevoir, j'ai développé une première version de l'affichage, voir l'[Annexe 29](#).

La page planning étant développée en React, elle est composée de composants imbriqués, les jours étant contenus dans les mois, eux-mêmes contenus dans le planning complet. J'avais initialement commencé par mettre la logique dans le composant Jour afin d'afficher les programmes prioritaires au bon jour. Cependant, cela signifiait que le calcul était effectué pour chaque jour, ce qui ralentissait considérablement le chargement. J'ai donc modifié ma méthode en intégrant la logique dans le calcul du planning complet, puis en transmettant les informations au mois, puis au jour via les Props. Cette approche s'est révélée plus rapide et plus légère.

Une fois la maquette reçue, voir l'[Annexe 30](#), j'ai commencé à travailler sur l'affichage, voir l'[Annexe 31](#). Après avoir résolu quelques petits problèmes de rendu, j'ai également développé la possibilité d'ouvrir la page de modification des programmes prioritaires en double-cliquant sur le programme dans le planning, en veillant à ce que cette fonctionnalité soit désactivée si aucun écran n'est connecté. Une fois toutes ces étapes réalisées, j'ai pu envoyer la fonctionnalité en préproduction afin que Camille effectue les tests nécessaires.

Synthèse des missions réalisées

Type	Nom du ticket	Temps de réalisation	Statut en fin du stage	Priorité
Fix	Correction du bug – page Planning	½ jour	En production	1
Feat	Copie des contenus	2 semaines	Approuvé – attente de mise en production	2
	Affichage des programmes prioritaires	3 jours	Approuvé – attente de mise en production	3
	Visualisation des mots de passes	1h	En production	4
	Barre de scroll	1 jour	En production	5
	Barre de recherche	1 jour	En production	6
	Tri des programmes	1h	En production	7
	Modification du nom des programmes	2 jours	Ajourné – en attente	8
	Ajout de nouvelles transitions	4 jours	A mettre en préproduction	9

Compétences acquises

Ce stage m'a permis de développer de nouvelles compétences techniques et professionnelles. Il m'a également aidée à mieux comprendre le fonctionnement de projets en entreprise et les exigences du monde du travail.

Compétences techniques

Au cours de ce stage, j'ai approfondi mes connaissances en développement, notamment en travaillant sur du code existant et en ajoutant de nouvelles fonctionnalités. J'ai appris à m'adapter à une architecture déjà en place et à intervenir sur un projet réel.

J'ai particulièrement développé mes compétences en développement web, notamment en HTML et surtout en TypeScript. J'ai découvert et appris à utiliser React et Tailwind CSS, ce qui m'a permis de mieux comprendre l'organisation d'une application en composants imbriqués et réutilisables.

J'ai également davantage travaillé avec GitHub, cette fois dans un contexte collaboratif. J'ai donc appris à gérer des branches, à résoudre des conflits et à respecter un workflow d'équipe.

Par ailleurs, j'ai travaillé avec Salesforce, un CRM que je connaissais déjà, mais que j'ai découvert sous un angle plus technique, notamment à travers sa base de données et les requêtes SOQL. Bien que la base de données de Lumméa soit relativement simple, j'ai également dû apprendre à effectuer des requêtes avec Prisma afin de communiquer correctement avec celle-ci.

Compétences organisationnelles

Bien qu'en tant que stagiaire je n'aie pas eu de délais stricts à respecter, j'ai appris à estimer mes temps de développement et à mieux planifier mon travail.

J'ai également gagné en autonomie et en organisation, notamment grâce à des outils comme Monday, sur lesquels les missions à réaliser étaient détaillées avec le nom des branches correspondantes. Cela m'a permis de mettre en œuvre une procédure adaptée aux aléas courants liés à mon activité.

Compétences relationnelles

Pour la première fois, j'ai travaillé en collaboration directe avec une cheffe de projet, ce qui a été une expérience très enrichissante. J'ai échangé régulièrement avec elle concernant l'avancement de mes tâches et les choix techniques effectués.

J'ai réalisé plusieurs démonstrations afin qu'elle valide mon travail avant de poursuivre. C'est également auprès d'elle que je communiquais mes estimations de temps de développement. Elle testait ensuite mes fonctionnalités et me faisait des retours constructifs, ce qui m'a permis d'améliorer la qualité de mon travail.

J'ai également participé à de nombreuses réunions avec différents membres de l'équipe. À certaines occasions, j'ai moi-même pris l'initiative d'organiser des réunions lorsque cela me semblait nécessaire, ce qui a représenté une prise de responsabilité importante et formatrice. J'ai ainsi appris à prendre la parole de manière pertinente lors des différentes interactions auxquelles j'ai été confronté.

Bilan Technique

Cette partie présente l'état d'avancement du projet à la fin de mon stage, en distinguant les travaux réalisés et ceux restant à finaliser.

Travaux réalisés

Sur le catalogue, la seule mission que j'ai pu finaliser concerne la gestion des déclinaisons, qui a été mise en production le dernier jour de mon stage.

Pour Lumméa, j'ai achevé plusieurs fonctionnalités importantes. L'œil de visualisation du mot de passe, la barre de scroll dans l'édition des contenus, le tri des programmes ainsi que la barre de recherche dans la partie *Lecteur* de la page *Administration* pour les administrateurs ont été mis en production le 21 janvier 2026. De plus, le bug de la page *Planning* a été résolu et déployé le 4 février 2026.

Travaux en cours

Certaines missions n'ont pas pu être complètement finalisées.

Sur le catalogue, la gestion des tarifs n'a pas été achevée par manque de temps et d'informations, mais j'ai laissé des indications précises à Cyriac pour qu'il puisse reprendre le travail.

Pour Lumméa, la plupart des missions sont terminées ou en phase finale de validation. La copie des contenus et l'affichage des programmes prioritaires ont été approuvés et seront bientôt mis en production. La barre de recherche côté revendeur est en préproduction et nécessite encore quelques tests. L'ajout des transitions est terminé côté développement, mais il reste à valider leur fonctionnement sur le moteur, en raison de particularités techniques. Enfin, la modification des noms des programmes n'a pas été réalisée car le ticket associé a été abandonné.

Conclusion

Ce stage m'a permis de mettre en pratique mes connaissances tout en développant de nouvelles compétences techniques et professionnelles. J'ai eu l'opportunité d'intervenir sur des projets concrets, de prendre des responsabilités et de mieux appréhender le fonctionnement d'une équipe de développement. Les missions qui m'ont été confiées ont ainsi contribué à l'avancement des projets, rendant cette expérience particulièrement enrichissante et formatrice.

J'ai également pris un réel plaisir à évoluer au sein du service IT de Splayce. L'équipe, très accueillante, m'a accompagnée tout au long de mon intégration et de ma montée en compétences. J'ai particulièrement apprécié travailler sur les différents projets et découvrir l'utilisation de React, qui a suscité chez moi un réel intérêt. Cette expérience a constitué une véritable révélation et a conforté mon envie de poursuivre dans cette voie professionnelle.

Internship Summary

As part of my second year of BTS Services Informatiques aux Organisations, I completed an internship at Splayce, a company specializing in healthcare communication solutions and part of the GDCOM Group. The main objective of this internship was to gain hands-on experience in software development within a professional environment.

During my internship, I contributed to two major projects. The first was an offline product catalog for the sales team, designed to provide access to product information even without internet. My main task was implementing a pricing system, aimed at improving usability and ensuring the sales team could quickly access accurate product data. I also enhanced the catalog's variation section, improving its layout and navigation with a search bar.

The second project, Lumméa, is a software solution used to manage display screens in pharmacies and town halls. I worked on multiple features, including password visibility, scroll bars, content sorting and searching, program prioritization, duplication of content for reseller accounts, and the addition of new display transitions. Some tasks required front-end adjustments with React and Tailwind CSS, while others involved back-end modifications, including database queries with Prisma and adjustments to the Electron engine. I also handled bug fixes, such as resolving issues on the planning page, and implementing user interface improvements based on feedback from the project manager.

This internship allowed me to strengthen my technical skills in TypeScript, React, Tailwind CSS, and database management, as well as my understanding of collaborative development with GitHub. I also improved my organizational skills, learning to estimate development times, prioritize tasks, and work autonomously while coordinating with the team. Moreover, I developed professional communication skills by interacting with the project manager, participating in meetings, and presenting demos to gather feedback.

Overall, this experience provided me with valuable insights into real-world software development, project management, and teamwork. It not only enhanced my technical abilities but also reinforced my interest in pursuing a career in IT and software development.

Annexes

dataset_2026-01-08_084207 > database.zip				
Trier Afficher Extraire tout				
Nom	Type	Taille compressée	Protégé pa...	Taille
categories.json	Fichier source JSON	12 Ko	Non	
errors.json	Fichier source JSON	11 Ko	Non	
families.json	Fichier source JSON	1 Ko	Non	
products.json	Fichier source JSON	1 775 Ko	Non	
search.json	Fichier source JSON	260 Ko	Non	
tarif.json	Fichier source JSON	318 Ko	Non	

Annexe 1 : dataset.zip avec le fichier tarif.json

Splayce Rechercher

Accueil > Catalogue > Reutilisables > Pp tisse & non tisse > Pp visuels std a personnaliser > **Sac pp tisse les sacs de josephine**

SAC JOSEPHINE A CARREAUX

ECO-SCORE **6** REF: JOSCARO400.0
stock disponible **3692**

Déclinaisons

- JOSCARO400.0 JOSCARO400.2
- JOSCARO400.1 JOSCOUO400.0
- JOSCOUO400.1



01 02 03 04

Description du produit
Détails de l'écorescore
Informations complémentaires
Grille

Annexe 2 : Produit parent

Splayce Rechercher

Accueil > Catalogue > Reutilisables > Pp tisse & non tisse > Pp visuels std a personnaliser > **Sac pp tisse les sacs de josephine**

SAC PP TISSE LES SACS DE JOSEPHINE

ECO-SCORE **6** REF: JOS---O400
stock disponible

Déclinaisons

- JOSCARO400.0 JOSCARO400.2
- JOSCARO400.1 JOSCOUO400.0
- JOSCOUO400.1



01 02 03

Description du produit
Détails de l'écorescore
Informations complémentaires

Annexe 3 : Produit enfant

Accueil > Catalogue > Reutilisables > Pp tisse & non tisse > Pp visuels std a personnaliser > **Sac pp tisse les sacs de josephine**

SAC JOSEPHINE A CARREAUX

ECO-SCORE **6** REF: JOSCARO400.2

stock disponible **3692**

% PRIX



Déclinaisons

- JOSCARO400.0 JOSCARO400.2
- JOSCARO400.1 JOSCOUO400.0
- JOSCOUO400.1

Description du produit **Détails de l'écorescore** **Informations complémentaires**

01 02 03

Grille

Tranche mini	Tranche maxi	Prix unitaire	Prix	Nombre de couleur
				•
				•
				•
				•
				•
				•

04

Annexe 4 : Version 1 de la grille tarifaire

Accueil > Catalogue > Reutilisables > Pp tisse & non tisse > Pp visuels std a personnaliser > **Sac pp tisse les sacs de josephine**

SAC JOSEPHINE A CARREAUX

ECO-SCORE **6** REF: JOSCARO400.2

stock disponible **3692**

% PRIX



Déclinaisons

- JOSCARO400.0 JOSCARO400.2
- JOSCARO400.1 JOSCOUO400.0
- JOSCOUO400.1

Description du produit **Détails de l'écorescore** **Informations complémentaires**

01 02 03

Grille

Type de prix	Coul							CLICHE
	•							

04

Annexe 5 : Version 2 de la grille tarifaire

ECO-SCORE **6** REF: 0878AQ

Sac en papier

Stock disponible: **11 825**
Dimensions: **400.100.100.320 MM**

Voir les déclinaisons



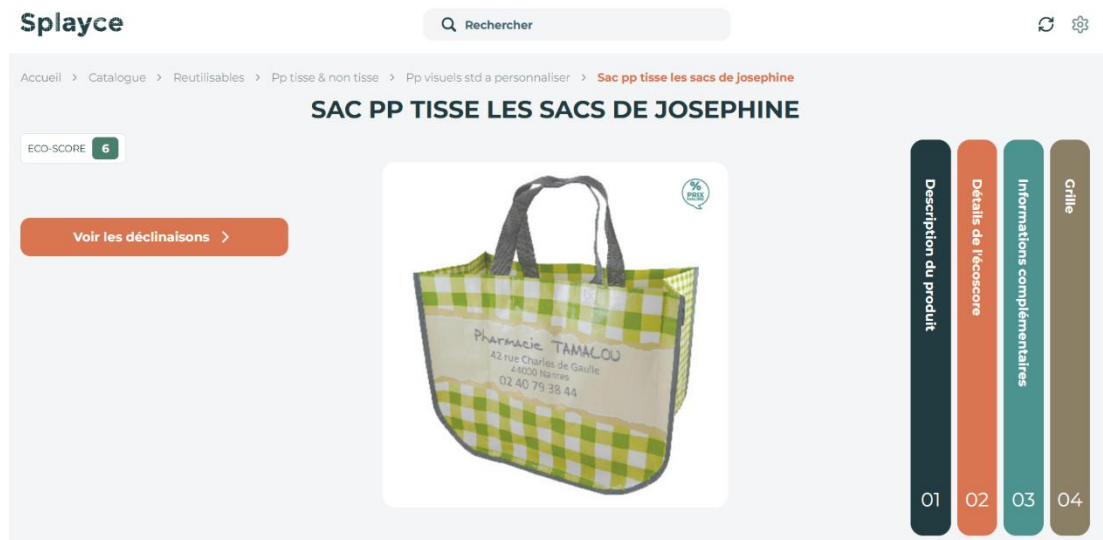
Déclinaisons

Text	Nom du produit	Référence
	JOSCARO400.2	Calendrier des Fêtes - sac en papier - 1 coul
	JOSCARO400.2	Calendrier des Fêtes - sac en papier - 1 coul
	JOSCARO400.2	Calendrier des Fêtes - sac en papier - 1 coul
	JOSCARO400.2	Calendrier des Fêtes - sac en papier - 1 coul
	JOSCARO400.2	Calendrier des Fêtes - sac en papier - 1 coul
	JOSCARO400.2	Calendrier des Fêtes - sac en papier - 1 coul
	JOSCARO400.2	Calendrier des Fêtes - sac en papier - 1 coul
	JOSCARO400.2	Calendrier des Fêtes - sac en papier - 1 coul

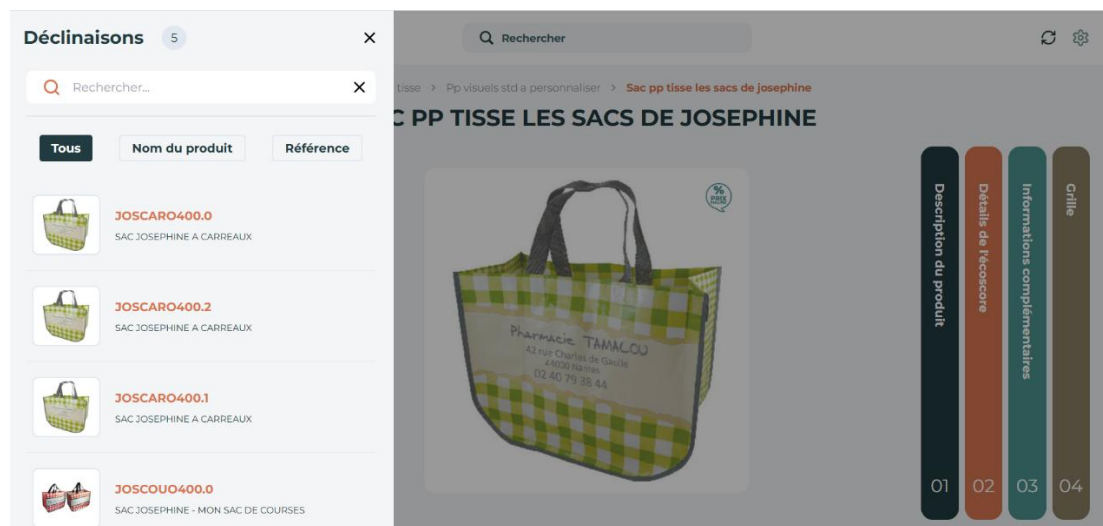
Description du produit **Détails de l'écorescore** **Informations complémentaires** **Grille tarifaire**

01 02 03 04

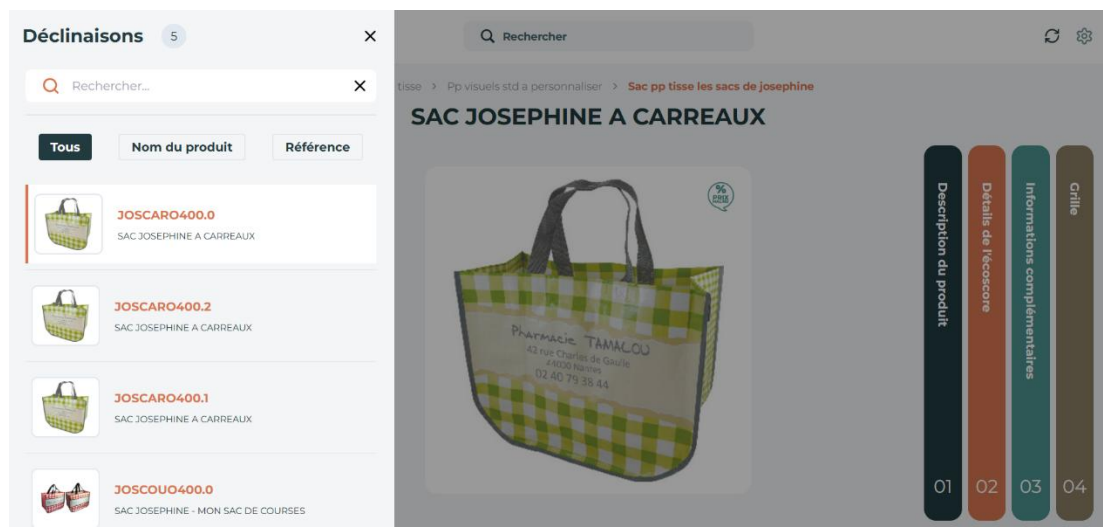
Annexe 6 : Maquettes des déclinaisons



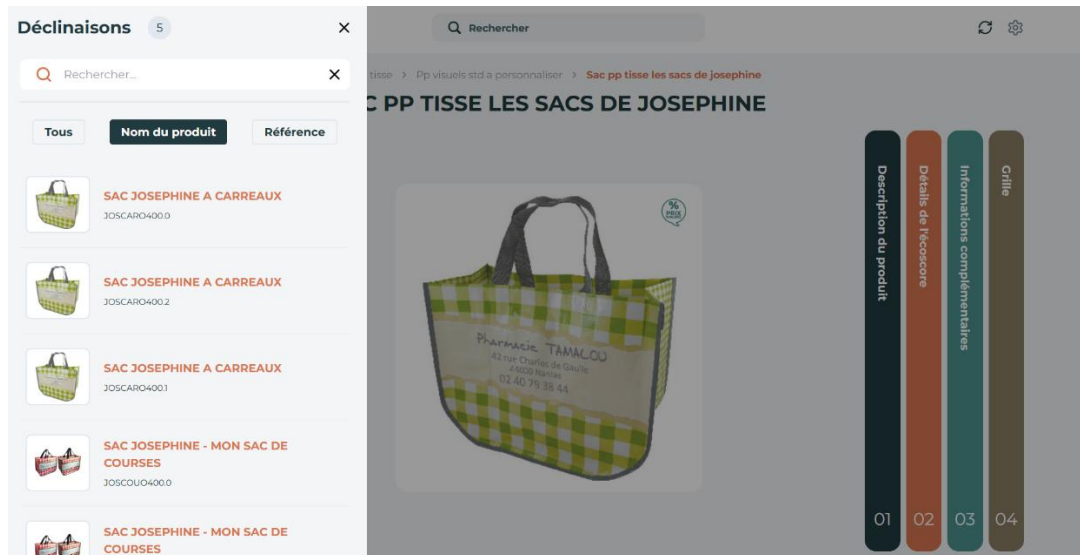
Annexe 7 : Bouton



Annexe 8 : Visuel final de la modale



Annexe 9 : Sélection d'une déclinaison



Annexe 10 : Filtre « Nom du produit » - inversion nom et référence

```
<div className="relative w-full max-w-sm">
  { !isMdpView ? (
    <input type="password" name="pwd" id="pwd" className="border-2 border-solid border-theme-50 w-full rounded-md p-2 m-2" placeholder={t('motDePasse')}
    <FaEye onClick={() => setIsMdpView(true)} className="absolute right-3 top-1/2 -translate-y-1/2 cursor-pointer"/>
  ) : (
    <input type="text" name="pwd" id="pwd" className="border-2 border-solid border-theme-50 w-full rounded-md p-2 m-2" placeholder={t('motDePasse')}
    <FaEyeLash onClick={() => setIsMdpView(false)} className="absolute right-3 top-1/2 -translate-y-1/2 cursor-pointer"/>
  )
}>
</div>
```

Annexe 11 : Code React

Bienvenue

Connectez-vous pour accéder à votre espace et profiter de nos services.

Email

nmerhand@splayce.eu

Mot de passe

Mdp

Connexion

Mot de passe oublié ?



Bienvenue

Connectez-vous pour accéder à votre espace et profiter de nos services.

Email

nmerhand@splayce.eu

Mot de passe

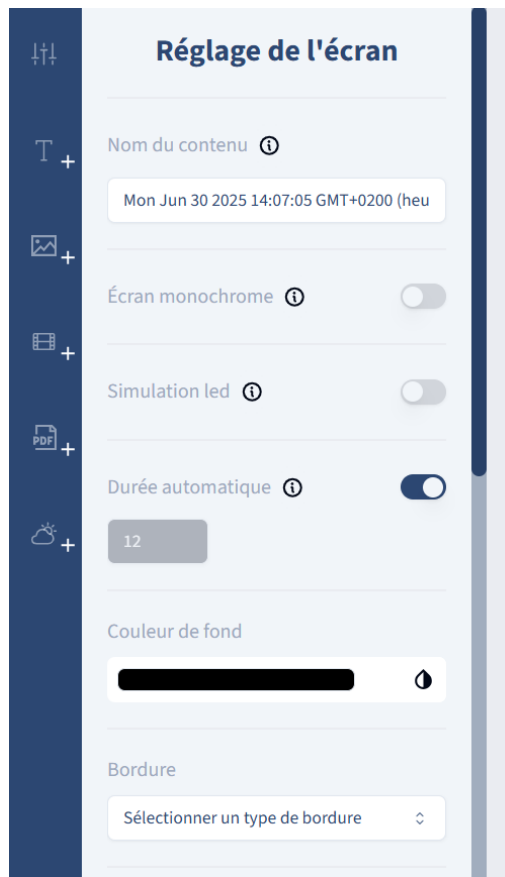
...

Connexion

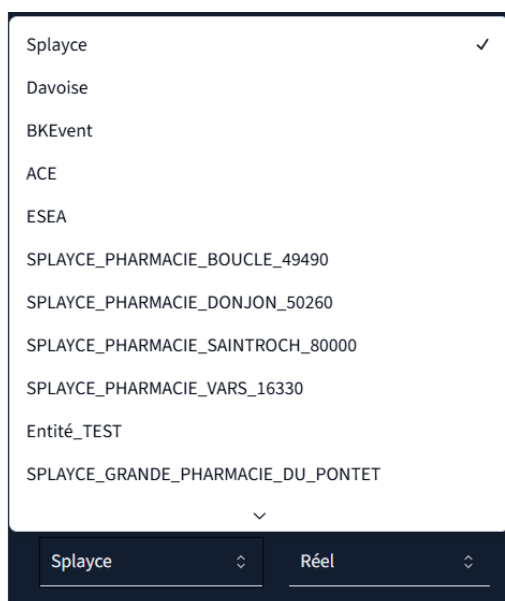
Mot de passe oublié ?



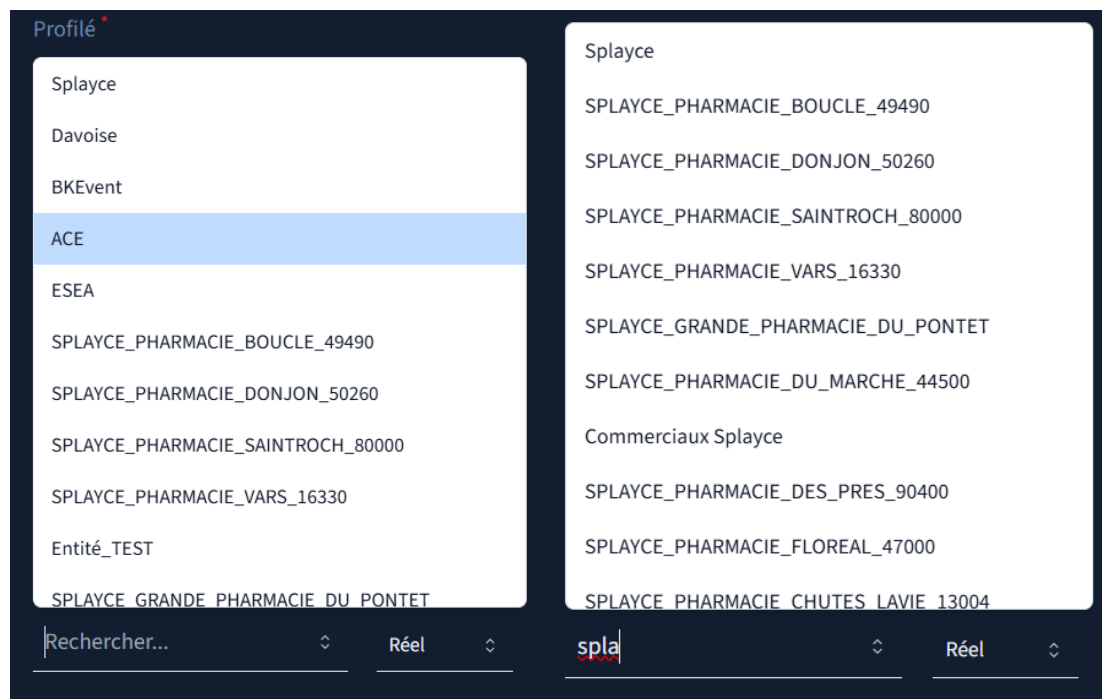
Annexe 12 : Vision conditionnelle du mot de passe



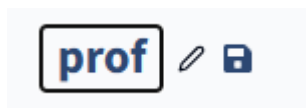
Annexe 13 : Barre de scroll dans la modification des contenus



Annexe 14 : Liste déroulante initiale – Avant modifications



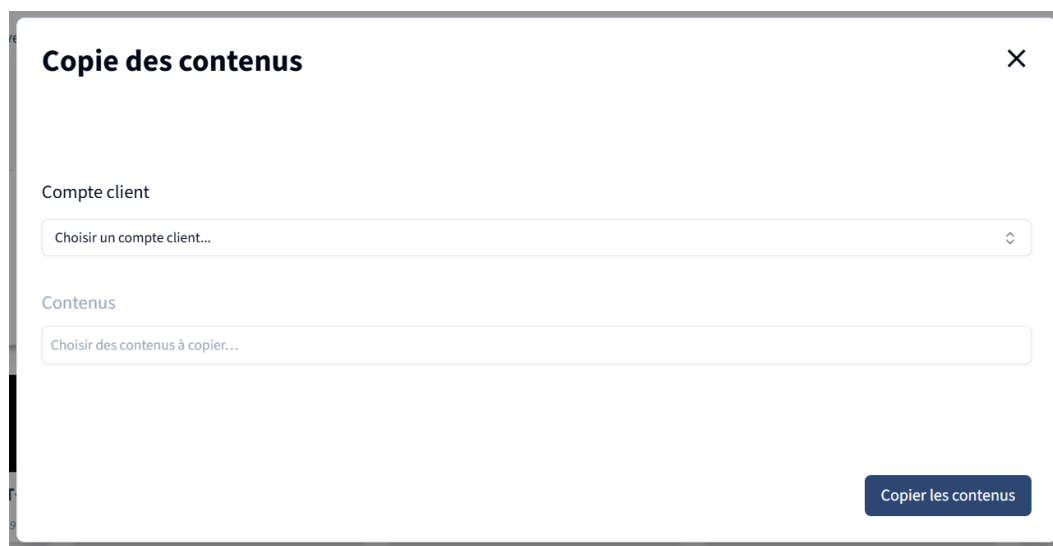
Annexe 15 : Liste déroulante avec barre de recherche – Après modifications



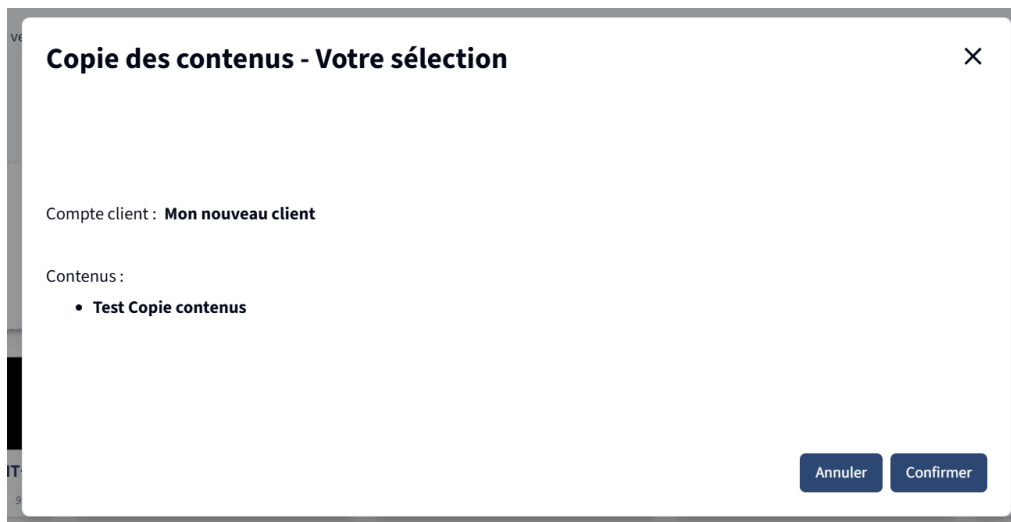
Annexe 16 : Nom du programme éditable



Annexe 17 : Modification du nom du programme dans la base de données



Annexe 18 : Version 1 de la copie des contenus – Première modale



Annexe 19 : Version 1 de la copie des contenus – Deuxième modale



Annexe 20 : Version 2 de la copie des contenus – Première modale



Annexe 21 : Version 2 de la copie des contenus – Deuxième modale

Compte cible	Contenus
BKEvent	OKAY
	sa va

Les contenus sélectionnés seront copiés vers le compte Client BKEvent. Ils ne seront pas automatiquement ajoutés à un programme ni planifiés.

Retour Confirmer le transfert

Annexe 22 : Version 2 de la copie des contenus – Troisième modale

Etape 1 - Sélection du compte cible
Le choix du compte permet de filtrer les contenus compatibles avec ses écrans.

Veuillez sélectionner un compte client

Compte client

Choisir un compte client...

Suivant

Etape 2 - Sélection des contenus
Les contenus sont filtrés en fonction du format d'écran présent sur le compte cible.

Veuillez sélectionner au moins un contenu

Contenus

0 contenus sélectionnés

Choisir des contenus à copier...

Retour Suivant

Annexe 23 : Version 2 de la copie des contenus – Messages d'erreurs

Etape 1 - Sélection du compte cible
Le choix du compte permet de filtrer les contenus compatibles avec ses écrans.

Compte client

Choisir un compte client...

Êtes-vous sûr de vouloir fermer cette fenêtre ?
Vos données ne seront pas sauvegardées

Retour Confirmer

Suivant

Annexe 24 : Version 2 de la copie des contenus – Fenêtre de confirmation de fermeture

✓ Les contenus ont été copiés avec succès.

Annexe 25 : Version 3 de la copie des contenus – Pop-up de confirmation

Choisir des contenus à copier...

Tout sélectionner

Mon Apr 07 2025 11:00:04 GMT+0200 (heure d'été d'Europe centrale)	LUXE
2	LUXE
128x128	LUXE

Choisir des contenus à copier...

Tout désélectionner

Mon Apr 07 2025 11:00:04 GMT+0200 (heure d'été d'Europe centrale)	✓
OKAY	✓
2	✓

Annexe 26 : Version 3 de la copie des contenus – Bouton 'Tout sélectionner/désélectionner'

```

async getAllContentByEntity(userEntityId: number, targetEntityId: number): Promise<any[]> {
  const screenTypes = await prisma.screen_type.findMany({
    where: {
      screens: {
        some: { entity_id: targetEntityId }
      }
    },
    select: { id: true }
  });

  if (screenTypes.length === 0) return [];

  const screenTypeIds = screenTypes.map(st => st.id);

  const contents = await prisma.content.findMany({
    where: {
      entity_id: userEntityId,
      type_id: { in: screenTypeIds }
    },
    select: {
      id: true,
      name: true,
      preview: {
        select: {
          url: true
        }
      },
      type: {
        select: {
          id: true,
          name: true
        }
      }
    }
  });

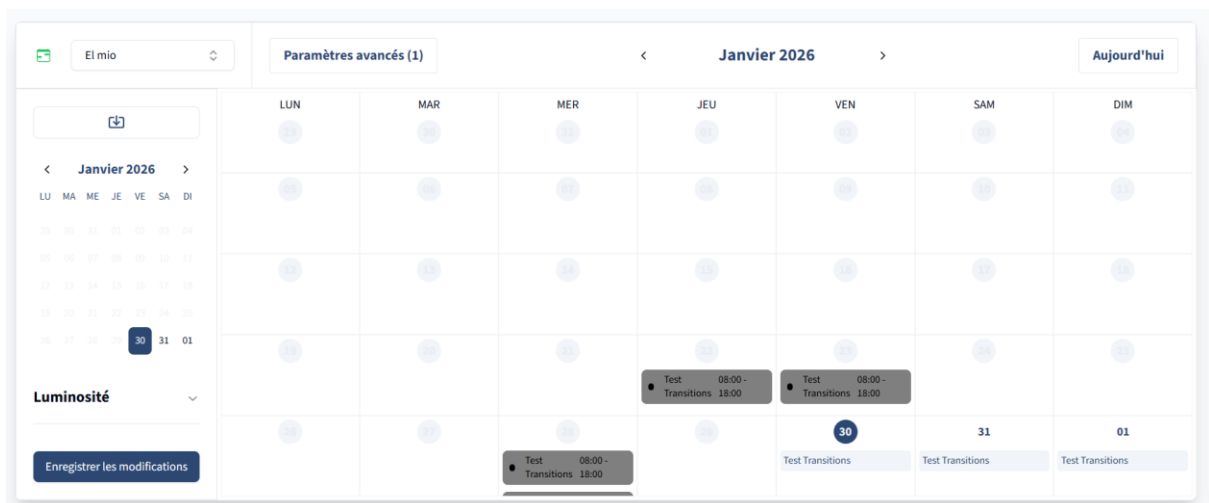
  return contents;
}

```

Annexe 27 : Requête Prisma finale avec le visuel des images et le type d'écran



Annexe 28 : Version 3 de la copie des contenus – Filtre par type d'écran



Annexe 29 : Version 1 de l'affichage des programmes prioritaires

Attention, ce compte n'est pas le votre

Rummée Contenus Programmes **Planning** Administration FR SPLAYCE_PHARMACIE_BOUCLE_49490

SPLAYCE_PHARMACIE_BOUCLE_49490 Paramètres avancés Affichage prioritaire Février 2026 Aujourd'hui

03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 01 02 03 04 05 06 07 08

Luminosité

Enregistrer les modifications

LUN	MAR	MER	JEU	VEN	SAM	DIM
02 ● JOURNALIER 09:00 - 12:30 ● JOURNALIER 14:31 - 19:15	03 ● JOURNALIER 09:00 - 12:30 ● JOURNALIER 14:31 - 19:15	04 ● Prioritaire : Octobre rose ● JOURNALIER 14:31 - 19:15	05 ● JOURNALIER 09:00 - 12:30 ● JOURNALIER 14:31 - 19:15	06 ● JOURNALIER 09:00 - 12:30 ● JOURNALIER 14:31 - 19:15	07 ● JOURNALIER 09:00 - 13:00	08
09 ● JOURNALIER 09:00 - 12:30 ● JOURNALIER 14:31 - 19:15	10 ● JOURNALIER 09:00 - 12:30 ● JOURNALIER 14:31 - 19:15	11 ● JOURNALIER 09:00 - 12:30 ● JOURNALIER 14:31 - 19:15 ● GARDE 19:15 - 08:45	12 ● JOURNALIER 09:00 - 12:30 ● JOURNALIER 14:31 - 19:15	13 ● JOURNALIER 09:00 - 12:30 ● JOURNALIER 14:31 - 19:15	14 ● Prioritaire : Octobre rose	15
16 ● JOURNALIER 09:00 - 12:30 ● JOURNALIER 14:31 - 19:15	17 ● JOURNALIER 09:00 - 12:30 ● JOURNALIER 14:31 - 19:15	18 ● JOURNALIER 09:00 - 12:30 ● JOURNALIER 14:31 - 19:15	19 ● Prioritaire : Octobre rose ● JOURNALIER 14:31 - 19:15	20 ● JOURNALIER 09:00 - 12:30 ● JOURNALIER 14:31 - 19:15	21 ● JOURNALIER 09:00 - 13:00	22
23 ● JOURNALIER 09:00 - 12:30 ● JOURNALIER 14:31 - 19:15	24 ● JOURNALIER 09:00 - 12:30 ● JOURNALIER 14:31 - 19:15	25 ● JOURNALIER 09:00 - 12:30 ● JOURNALIER 14:31 - 19:15	26 ● JOURNALIER 09:00 - 12:30 ● JOURNALIER 14:31 - 19:15	27 ● JOURNALIER 09:00 - 12:30 ● JOURNALIER 14:31 - 19:15	28 ● JOURNALIER 09:00 - 13:00	01
02 ● JOURNALIER 09:00 - 12:30 ● JOURNALIER 14:31 - 19:15	03 ● JOURNALIER 09:00 - 12:30 ● JOURNALIER 14:31 - 19:15	04 ● JOURNALIER 09:00 - 12:30 ● JOURNALIER 14:31 - 19:15	05 ● JOURNALIER 09:00 - 12:30 ● JOURNALIER 14:31 - 19:15	06 ● JOURNALIER 09:00 - 12:30 ● JOURNALIER 14:31 - 19:15	07 ● JOURNALIER 09:00 - 13:00	08

Annexe 30 : Maquette de l'affichage des programmes prioritaires

El mio Paramètres avancés (2) Février 2026 Aujourd'hui

03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 01 02 03 04 05 06 07 08

Luminosité

Enregistrer les modifications

LUN	MAR	MER	JEU	VEN	SAM	DIM
02	03 ● Prioritaire : Test	04 ● Prioritaire : Test	05 ● Prioritaire : Test	06 ● Prioritaire : Test	07 ● Prioritaire : Test	08
09	10	11	12	13	14	15
16	17	18 ● Test 08:00 - 18:00	19 ● Prioritaire : Test Transitions	20	21	22
23	24	25	26	27	28	01
02	03	04	05	06	07	08

Annexe 31 : Version final de l'affichage des programmes prioritaires